Java

# Manejo de Errores

Exception · Error · try-catch · finally · try-with-resources · @ExceptionHandler

Brandon Martínez · NTT DATA

JERARQUÍA

# Árbol de Throwable

Todo en Java que se puede lanzar hereda de Throwable

**Checked:** verificadas en compilación **Unchecked (Runtime):** no obligatorias — errores de ejecución

# Error vs Exception

## ⚠ Error

- Problemas graves del sistema o JVM
- NO se deben capturar ni manejar
- Irrecuperables en la mayoría de casos
- Causados por el entorno de ejecución
- Ejemplo: OutOfMemoryError cuando la JVM se queda sin memoria heap

VS

## ✓ Exception

- Situaciones anómalas pero manejables
- SE pueden y DEBEN capturar
- Recuperables con lógica apropiada
- Causadas por el código de la aplicación
- Ejemplo: FileNotFoundException al intentar leer un archivo inexistente

ERROR

# Hijos principales de Error

Nunca intentes capturarlos con catch — son señales de fallo crítico

## OutOfMemoryError

La JVM no puede asignar más memoria heap. Ocurre al crear demasiados objetos sin liberar referencias.

```
// Causa típica
List<byte[]> list = new ArrayList<>();
while (true) list.add(new byte[1024 * 1024]);
```

## StackOverflowError

La pila de llamadas se desbordó. Generalmente causado por recursión infinita o muy profunda.

```
// Recursión sin caso base
void recursive() {
    recursive(); // StackOverflowError
}
```

## NoClassDefFoundError

La JVM no encontró una clase en tiempo de ejecución que sí existía al compilar.

```
// Clase presente al compilar
// pero ausente en el classpath al ejecutar
```

## AssertionError

Una aserción (assert) falló. Se usa para invariantes durante desarrollo.

```
int x = -5;
assert x >= 0 : "x debe ser positivo";
```

# Hijos principales de Exception

## ✓ Checked Exceptions (deben declararse o capturarse)

### IOException

Error de entrada/salida: archivos, streams, conexiones de red.

### SQLException

Error en operaciones con bases de datos JDBC.

### ClassNotFoundException

Class.forName() no encontró la clase solicitada.

### ParseException

Error al parsear texto (fechas, formatos).

## ✗ Unchecked Exceptions (RuntimeException — no obligatorias)

### NullPointerException

Acceso a un objeto con referencia null.

### IllegalArgumentException

Argumento inválido pasado a un método.

### ArrayIndexOutOfBoundsException

Índice fuera del rango del arreglo.

### ClassCastException

Cast inválido entre tipos incompatibles.

# try-catch y multi-catch

## Básico

```
try {
    String s = null;
    s.length();  // NullPointerException
} catch (NullPointerException e) {
    System.out.println("Referencia nula: " +
e.getMessage());
}
```

## Multi-catch (Java 7+)

```
try {
    riskyOperation();
} catch (IOException | SQLException e) {
    log.error("Error IO o SQL", e);
} catch (Exception e) {
    throw new RuntimeException(e);
}
```

## Buenas prácticas

✓ Captura la excepción más específica primero

✓ Nunca captures Exception genérica sin relanzar

✗ Evita bloques catch vacíos — silencian errores

✗ No uses excepciones para flujo de control normal

# El bloque finally

El bloque finally SIEMPRE se ejecuta, sin importar si hubo excepción o no.
Es ideal para liberar recursos: conexiones, archivos, locks.

## Estructura completa try / catch / finally

```
Connection conn = null;
try {
    conn = dataSource.getConnection();
    // ... operaciones con la BD
} catch (SQLException e) {
    log.error("Error en BD", e);
} finally {
    if (conn != null) conn.close(); // Siempre se libera
}
```

### ⚡ Siempre ejecutado

Incluso si hay return en el try o el catch.

### ⚠ Excepción en finally

Si lanzas excepción en finally, oculta la original.

### 💡 Reemplazado por

try-with-resources es preferido desde Java 7.